Lab 4 — Câu lệnh điều kiện và vòng lặp

1. Đề bài tiếng Anh

LAB Session 4

PART 1: CONDITIONAL STATEMENT

Exercise 1 — Sorting numbers

Write a program that inputs three real numbers and prints them in ascending order.

Requirement: Use only `if...else` statements.

Example: Input: `8.5 3.2 6.7` → Output: `3.2 6.7 8.5`

Exercise 2 — Quarter of the year

Write a program that inputs a month (1–12) and determines which quarter of the year it belongs to.

Example: Input: `5` → Output: `Quarter 2`

Requirement: Use `if...elif...else` or `match...case`.

Exercise 3 — Days in a month

Write a program that inputs a month. Determine the number of days in that month.

Example: Input: `3` → Output: `31 days`

Exercise 4 — Leap year

Write a program that inputs a year and determines whether it is a leap year.

Leap year rules:

- A year divisible by 400 is a leap year.
- A year divisible by 4 but not divisible by 100 is a leap year.
- Otherwise, it is not a leap year.

Examples: `2024` → `Leap year`; `2023` → `Not a leap year`.

Exercise 5 — Number name (two digits)

Write a program that inputs a two-digit integer and prints its number name.

Examples: `12` → `muoi hai`; `25` → `hai muoi lam`.

Exercise 6 — Number name (three digits)

Write a program that inputs a three-digit integer and prints its number name.

Example: `123` → `mot tram hai muoi ba`.

Exercise 7 — Electricity bill

Write a program that inputs the amount of electricity consumed in kWh and calculates the electricity bill based on the following rates:

Electricity usage	Price
0–50 kWh	1,800 VND/kWh
51–100 kWh	2,000 VND/kWh
101–200 kWh	2,500 VND/kWh
Above 200 kWh	3,000 VND/kWh

Example in original Word: Input `120` → Output `282000 VND`.

> **Ghi chú hiệu đính:** Nếu giá điện được tính **lũy tiến từng bậc**, 120 kWh cho kết quả **240,000 VND**. Ví dụ `282000` trong Word không khớp bảng giá; `SOLUTION.py` tính lũy tiến và có kiểm tra kết quả. Số liệu ví dụ gốc được giữ ở đây để đối chiếu.

PART 2: LOOP

Exercise 8 — Sum from 1 to n

Input a positive integer `n` and calculate the sum of all integers from 1 to `n`.

Example: Input `10` → `Sum = 55`.

Exercise 9 — Count digits

Input a positive integer and count the number of digits.

Example: `123456` → `Number of digits = 6`.

Requirement: Use a `while` loop.

Exercise 10 — Prime number

Input a positive integer and determine whether it is prime.

Example: `17` → `17 is a prime number.`

Exercise 11 — GCD

Input two positive integers and find their Greatest Common Divisor (GCD).

Example: `24 36` → `GCD = 12`.

Exercise 12 — Rectangle pattern

Input integers `m` and `n`, then print a rectangle with `m` rows and `n` columns using `*`.

For `m = 4`, `n = 6`:

```
******
******
******
******
```

Requirement: Use nested loops.

Exercise 13 — Repeated-digit triangle

Input an integer `n` and print a number triangle. For `n = 5`:

```
1
22
333
4444
55555
```

Requirement: Use nested loops.

Exercise 14 — Consecutive-number triangle

Input an integer `n` and print a triangle containing consecutive numbers. For `n = 4`:

```
1
2 3
4 5 6
7 8 9 10
```

Requirement: Use nested loops and a variable to track the current number.

Đề được chuyển từ `LAB Session 4.docx` sang Markdown; các ví dụ được đặt trong khối văn bản để không lỗi định dạng.

2. Đề bài tiếng Việt

LAB SESSION 4 — Câu lệnh điều kiện và vòng lặp

PHẦN 1: CÂU LỆNH ĐIỀU KIỆN

Bài 1 — Sắp xếp ba số Nhập ba số thực và in ra theo thứ tự tăng dần. **Chỉ được dùng `if...else`**, không dùng hàm sắp xếp có sẵn. Ví dụ: `8.5 3.2 6.7` → `3.2 6.7 8.5`.

Bài 2 — Quý trong năm Nhập tháng từ 1–12 rồi xác định thuộc quý nào. Ví dụ: 5 → Quý 2. **Dùng `if...elif...else` hoặc `match...case`**.

Bài 3 — Số ngày trong tháng Nhập tháng và xác định số ngày trong tháng. Ví dụ tháng 3 có 31 ngày. Nếu xét tháng 2 và có năm, cần xét năm nhuận.

Bài 4 — Năm nhuận Nhập năm rồi xác định năm nhuận. Năm chia hết cho 400 là năm nhuận; hoặc chia hết cho 4 nhưng không chia hết cho 100. Các trường hợp khác không nhuận. Ví dụ 2024 nhuận, 2023 không nhuận.

Bài 5 — Đọc số có hai chữ số Nhập số nguyên hai chữ số, in cách đọc số bằng tiếng Việt. Ví dụ: 12 → `muoi hai`; 25 → `hai muoi lam`.

Bài 6 — Đọc số có ba chữ số Nhập số nguyên ba chữ số, in cách đọc bằng tiếng Việt. Ví dụ: 123 → `mot tram hai muoi ba`.

Bài 7 — Tính tiền điện Nhập lượng điện tiêu thụ (kWh), tính hóa đơn từ bảng giá:

Lượng điện tiêu thụ	Đơn giá
0–50 kWh	1.800 VND/kWh
51–100 kWh	2.000 VND/kWh
101–200 kWh	2.500 VND/kWh
Trên 200 kWh	3.000 VND/kWh

Ví dụ trong Word: 120 kWh → 282.000 VND. *Giá trị này được giữ nguyên khi dịch; lưu ý kiểm tra lại với phương pháp tính lũy tiến.*

PHẦN 2: VÒNG LẶP

Bài 8 — Tổng từ 1 đến n Nhập số nguyên dương `n`, tính tổng các số nguyên từ 1 đến `n`. Ví dụ: 10 → 55.

Bài 9 — Đếm chữ số Nhập số nguyên dương, **dùng `while`** đếm số chữ số. Ví dụ 123456 có 6 chữ số.

Bài 10 — Kiểm tra số nguyên tố Nhập số nguyên dương và xác định có phải số nguyên tố không. Ví dụ 17 là số nguyên tố.

Bài 11 — Ước chung lớn nhất Nhập hai số nguyên dương, tính GCD. Ví dụ 24 và 36 → 12.

Bài 12 — Hình chữ nhật dấu sao Nhập `m`, `n`, dùng **vòng lặp lồng nhau** in hình chữ nhật `m` hàng, `n` cột bằng ký tự `*`. Với `m=4,n=6`:

```
******
******
******
******
```

Bài 13 — Tam giác chữ số lặp Nhập `n`, dùng **vòng lặp lồng nhau** in tam giác. Với `n=5`:

```
1
22
333
4444
55555
```

Bài 14 — Tam giác số liên tiếp Nhập `n`, dùng **vòng lặp lồng nhau** và biến đếm liên tục. Với `n=4`:

```
1
2 3
4 5 6
7 8 9 10
```

3. Code Python có chú thích

```
from math import isqrt  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.

def task01_sort_three(a, b, c):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if a > b: a, b = b, a  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if b > c: b, c = c, b  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if a > b: a, b = b, a  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return a, b, c  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task02_quarter(month):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if not 1 <= month <= 12: raise ValueError("Month must be 1..12")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if month <= 3: return 1  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    elif month <= 6: return 2  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
    elif month <= 9: return 3  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
    else: return 4  # Xử lý trường hợp ngược lại hoặc còn lại.

def leap_year(year):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task03_month_days(month, year=2026):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    days = [31,28,31,30,31,30,31,31,30,31,30,31]  # Tính hoặc cập nhật biến days theo công thức của bài.
    if not 1 <= month <= 12: raise ValueError("Month must be 1..12")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return 29 if month == 2 and leap_year(year) else days[month-1]  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task04_is_leap(year):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if year < 1: raise ValueError("Year must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return leap_year(year)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

WORDS = ["khong","mot","hai","ba","bon","nam","sau","bay","tam","chin"]  # Tính hoặc cập nhật biến WORDS theo công thức của bài.

def task05_two_digit(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if not 10 <= n <= 99: raise ValueError("Expected two-digit integer")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    tens, unit = divmod(n,10)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    if tens == 1:  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
        head = "muoi"  # Tính hoặc cập nhật biến head theo công thức của bài.
    else:  # Xử lý trường hợp ngược lại hoặc còn lại.
        head = WORDS[tens] + " muoi"  # Tính hoặc cập nhật biến head theo công thức của bài.
    if unit == 0: return head  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if unit == 5: return head + " lam"  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if unit == 1 and tens > 1: return head + " mot"  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return head + " " + WORDS[unit]  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task06_three_digit(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if not 100 <= n <= 999: raise ValueError("Expected three-digit integer")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    hundred, rest = divmod(n,100)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    head = WORDS[hundred] + " tram"  # Tính hoặc cập nhật biến head theo công thức của bài.
    if rest == 0: return head  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    if rest < 10:  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
        return head + " linh " + ("nam" if rest == 5 else WORDS[rest])  # Trả kết quả (return: trả về) cho nơi gọi hàm.
    return head + " " + task05_two_digit(rest)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task07_electricity_bill(kwh):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if kwh < 0: raise ValueError("Usage cannot be negative")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    remaining = float(kwh)  # Tính hoặc cập nhật biến remaining theo công thức của bài.
    total = 0.0  # Tính hoặc cập nhật biến total theo công thức của bài.
    for width, rate in ((50,1800),(50,2000),(100,2500)):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
```

```
        amount = min(remaining,width)  # Tính hoặc cập nhật biến amount theo công thức của bài.
        total += amount * rate  # Tính hoặc cập nhật biến total theo công thức của bài.
        remaining -= amount  # Tính hoặc cập nhật biến remaining theo công thức của bài.
        if remaining <= 0: return total  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return total + remaining * 3000  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task08_sum_to_n(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if n <= 0: raise ValueError("n must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    total=0  # Tính hoặc cập nhật biến total theo công thức của bài.
    for value in range(1,n+1): total += value  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
    return total  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task09_count_digits(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if n <= 0: raise ValueError("n must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    count=0  # Tính hoặc cập nhật biến count theo công thức của bài.
    while n:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        count += 1  # Tính hoặc cập nhật biến count theo công thức của bài.
        n //= 10  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    return count  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task10_is_prime(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if n <= 1: return False  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    for divisor in range(2,isqrt(n)+1):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        if n % divisor == 0: return False  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return True  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task11_gcd(a,b):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if a<=0 or b<=0: raise ValueError("Both numbers must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    while b:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        a,b=b,a%b  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    return a  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task12_rectangle(rows,cols):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if rows <= 0 or cols <= 0: raise ValueError("Rows/cols must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    output=[]  # Tính hoặc cập nhật biến output theo công thức của bài.
    for _ in range(rows):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        line=""  # Tính hoặc cập nhật biến line theo công thức của bài.
        for _ in range(cols):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
            line += "*"  # Tính hoặc cập nhật biến line theo công thức của bài.
        output.append(line)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(output)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task13_repeated_triangle(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if n <= 0: raise ValueError("n must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    output=[]  # Tính hoặc cập nhật biến output theo công thức của bài.
    for i in range(1,n+1):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        line=""  # Tính hoặc cập nhật biến line theo công thức của bài.
        for _ in range(i):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
            line += str(i)  # Tính hoặc cập nhật biến line theo công thức của bài.
        output.append(line)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(output)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def task14_consecutive_triangle(n):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if n <= 0: raise ValueError("n must be positive")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    rows=[]  # Tính hoặc cập nhật biến rows theo công thức của bài.
    current=1  # Tính hoặc cập nhật biến current theo công thức của bài.
    for i in range(1,n+1):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        values=[]  # Tính hoặc cập nhật biến values theo công thức của bài.
```

```
        for _ in range(i):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
            values.append(str(current))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
            current += 1  # Tính hoặc cập nhật biến current theo công thức của bài.
        rows.append(" ".join(values))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return "\n".join(rows)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def main():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if not (task01_sort_three(8.5,3.2,6.7)==(3.2,6.7,8.5)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task02_quarter(5)==2): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task03_month_days(2,2024)==29): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task04_is_leap(2024) and not task04_is_leap(2023)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task05_two_digit(25)=="hai muoi lam"): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task06_three_digit(123)=="mot tram hai muoi ba"): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task07_electricity_bill(120)==240000): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task08_sum_to_n(10)==55): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task09_count_digits(123456)==6): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task10_is_prime(17) and not task10_is_prime(1)): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task11_gcd(24,36)==12): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    if not (task12_rectangle(4,6)=="******\n******\n******\n******"): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ
ràng.
    if not (task13_repeated_triangle(5)=="1\n22\n333\n4444\n55555"): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ
ràng.
    if not (task14_consecutive_triangle(4)=="1\n2 3\n4 5 6\n7 8 9 10"): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ
ràng.
    print("PASS: 14 exercises")  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Example electricity 120 kWh:",task07_electricity_bill(120),"VND")  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Pattern:",task14_consecutive_triangle(4),sep="\n")  # In kết quả (output: kết quả) để người học đối chiếu.

if __name__=="__main__":  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    main()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
```

4. Giải thích nội dung và cách giải

Câu 1

So sánh cặp `(a,b)`, `(b,c)`, rồi `(a,b)` và đổi chỗ khi cặp không đúng thứ tự. Đây là cách dùng điều kiện, không gọi `sort()`.

Vì sao so sánh từng cặp? Đề yêu cầu luyện `if/else`, nên không dùng hàm sắp xếp có sẵn. Đưa số nhỏ về đầu bằng các lần so sánh và hoán đổi. **Quy trình:** So a với b → so b với c → so lại a với b. Sau lượt cuối, thứ tự là tăng dần.

```
a, b, c = 8.5, 3.2, 6.7
if a > b: a, b = b, a
if b > c: b, c = c, b
if a > b: a, b = b, a
print(a, b, c)  # 3.2 6.7 8.5
```

Cú pháp: > so sánh lớn hơn; `a, b = b, a` hoán đổi hai biến mà không cần biến tạm. Không dùng `sorted` vì trái mục đích luyện điều kiện.

Điểm cần kiểm tra: dùng `if/elif/else` đúng các bài điều kiện và so sánh hai số.

Ví dụ và lỗi thường gặp: Cho ba giá trị 8,5; 3,2; 6,7, lần lượt đổi vị trí theo phép so sánh. Cuối cùng cần có 3,2 ≤ 6,7 ≤ 8,5. Không gọi sorted() vì đề yêu cầu luyện điều kiện.

Thuật ngữ cần nhớ: ascending (tăng dần).

Câu 2

Kiểm tra tháng hợp lệ, sau đó xem tháng nằm ở 1–3, 4–6, 7–9 hay 10–12 để trả quý.

Vì sao chia thành bốn mốc? Một năm có 12 tháng và mỗi quý gồm ba tháng; điều kiện theo mốc dễ đọc và đúng yêu cầu luyện nhánh. **Quy trình:** Kiểm tra tháng 1–12 → xét đến 3, đến 6, đến 9 → còn lại quý 4.

```
month = 5
if not 1 <= month <= 12:
    raise ValueError("Tháng không hợp lệ")
quarter = 1 if month <= 3 else 2 if month <= 6 else 3 if month <= 9 else 4
```

Cú pháp: <= có tính cả mốc; `not` phủ định điều kiện hợp lệ; `if/elif/else` trong code hoàn chỉnh giúp tránh tính mốc sai. Có thể viết `(month - 1) // 3 + 1` nhưng không dùng nó để thay bài luyện điều kiện; `//` là chia lấy phần nguyên.

Điểm cần kiểm tra: quy tắc năm nhuận và số ngày tháng 2.

Ví dụ và lỗi thường gặp: Chia 12 tháng thành bốn nhóm ba tháng; tháng 5 thuộc quý 2. Kiểm tra đầu vào trước vì tháng 0 và 13 không hợp lệ.

Thuật ngữ cần nhớ: quarter (quý).

Câu 3

Dùng danh sách ngày của 12 tháng. Nếu tháng 2 của năm nhuận thì lấy 29, không lấy 28. Mã này tham chiếu thêm `leap_year()` ở mã nguồn đã học.

Vì sao cần biết năm? Tháng 2 có thể có 28 hoặc 29 ngày, nên chỉ nhập tháng chưa đủ nếu muốn xác định chính xác tháng 2; code tham khảo dùng tham số `year=2026` khi không truyền năm. **Quy trình:** Kiểm tra tháng → lấy số ngày mặc định từ danh sách → riêng tháng 2 kiểm tra năm nhuận.

```
days = [31,28,31,30,31,30,31,31,30,31,30,31]
month, year = 2, 2024
result = 29 if month == 2 and leap_year(year) else days[month - 1]
```

Giải thích: `days[month - 1]` đổi tháng 1 thành chỉ số 0; `and` yêu cầu cả tháng 2 và năm nhuận; `year=2026` là **tham số mặc định** của hàm cuối bài, không có nghĩa mọi năm là 2026.

Điểm cần kiểm tra: tách chữ số bằng phép chia nguyên `//` và chia dư `%`.

Ví dụ và lỗi thường gặp: Tháng 2 năm 2024 có 29 ngày, năm 2023 có 28 ngày. Nên tách quy tắc năm nhuận thành hàm để dễ kiểm tra.

Thuật ngữ cần nhớ: month (tháng).

Câu 4

Dùng toán tử `%` kiểm tra các trường hợp chia hết; điều kiện năm dương loại đầu vào không hợp lệ.

Vì sao kiểm tra 400, 100 và 4? Năm chia hết cho 4 thường là nhuận, nhưng năm tròn thế kỷ chỉ nhuận khi chia hết cho 400. Bỏ điều kiện 100 sẽ khiến năm 1900 bị xử lý sai. **Quy trình:** Kiểm tra năm dương → xét chia hết 400 → hoặc chia hết 4 nhưng không chia hết 100.

```
year = 2024
is_leap = (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0)
```

Cú pháp: `%` lấy số dư; `== 0` là chia hết; `!=` là khác; `and` cần cả hai điều kiện; `or` cho phép một trong hai trường hợp năm nhuận. Ví dụ 2000 đúng, 1900 sai.

Điểm cần kiểm tra: tiền điện lũy tiến: `50×1800 + 50×2000 + 20×2500 = 240000 VND` cho 120 kWh; đề Word ghi 282000, không khớp.

Ví dụ và lỗi thường gặp: Năm 1900 không nhuận, năm 2000 nhuận. Thử hai năm này để phát hiện lỗi chỉ kiểm tra chia hết cho 4.

Thuật ngữ cần nhớ: leap year (năm nhuận).

Câu 5

Tách hàng chục và đơn vị, chọn từ đọc thích hợp trong danh sách từ đã có (`WORDS`). Một số trường hợp đặc biệt như số 25 đọc `lam`.

Vì sao tách hàng chục và hàng đơn vị? Muốn đọc một số hai chữ số phải biết chữ số hàng chục và phần còn lại để ghép từ theo tiếng Việt. **Quy trình:** Kiểm tra 10–99 → chia thành chục và đơn vị → chọn trường hợp 10–19, chục khác 1, các âm đọc đặc biệt → ghép chuỗi.

```
n = 25
tens, units = divmod(n, 10)  # 2 và 5
print(task05_two_digit(n))  # Hàm hoàn chỉnh xử lý cách đọc số
```

Cú pháp: `divmod(n, 10)` trả cùng lúc thương nguyên và số dư, tương đương `n // 10` và `n % 10`. `WORDS[index]` tra từ đọc số; `return` trả chuỗi, không phải phép cộng số.

Điểm cần kiểm tra: vòng lặp `for`/`while`, nguyên tố đến √n, Euclid `gcd(a,b)=gcd(b,a%b)`.

Ví dụ và lỗi thường gặp: Đối với hai chữ số, dùng //10 lấy hàng chục, %10 lấy hàng đơn vị. Phát âm số 5 ở hàng đơn vị cần theo quy tắc tiếng Việt.

Thuật ngữ cần nhớ: digit (chữ số).

Câu 6

Tách hàng trăm và phần dư. Nếu phần dư nhỏ hơn 10 dùng cách đọc `linh`, còn lại gọi lại hàm đọc hai chữ số; cần dùng `WORDS` và `task05_two_digit` đã được định nghĩa chung.

Vì sao dùng lại bài hai chữ số? Đọc số ba chữ số gồm hàng trăm và phần còn lại; tái sử dụng hàm đọc hai chữ số tránh viết lại toàn bộ quy tắc. **Quy trình:** Kiểm tra 100–999 → tách trăm và phần dư → đọc trăm → nếu dư bằng 0 kết thúc, dư dưới 10 thêm “linh”, còn lại gọi hàm hai chữ số.

```
n = 105
hundreds, rest = divmod(n, 100)  # 1 và 5
result = WORDS[hundreds] + " tram"
if 0 < rest < 10:
    result += " linh " + WORDS[rest]
```

Cú pháp: `divmod(n,100)` tách trăm và phần dư; `+` nối chuỗi; `0 < rest < 10` xét khoảng. Trong code hoàn chỉnh, chữ số 5 có quy tắc đọc riêng, và phần dư hai chữ số được chuyển sang `task05_two_digit`.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Chia số nguyên cho 100 tìm trăm; xét phần dư để đọc chục và đơn vị. Ví dụ 105 cần cách đọc 'linh'.

Thuật ngữ cần nhớ: hundreds (hàng trăm).

Câu 7

Không nhân tất cả kWh với mức giá cao nhất. Lần lượt tính 50 kWh ở bậc 1, 50 kWh tiếp theo bậc 2, phần còn lại 20 kWh bậc 3.

Vì sao tính theo các bậc? Giá mỗi kWh thay đổi theo lượng điện tích lũy. Không thể lấy cả 120 kWh nhân đơn giá 2.500 vì 100 kWh đầu có giá thấp hơn. **Quy trình:** Kiểm tra lượng điện không âm → lấy tối đa 50 kWh cho bậc 1 → tối đa 50 cho bậc 2 → tối đa 100 cho bậc 3 → phần còn lại ở bậc 4 → cộng chi phí.

```
remaining = 120
total = 0
for width, rate in [(50,1800),(50,2000),(100,2500)]:
    amount = min(remaining, width)
    total += amount * rate
    remaining -= amount
```

```
    if remaining <= 0:
        break
total += remaining * 3000
print(total)  # 240000
```

Cú pháp: `min` lấy mức điện chưa dùng hoặc độ rộng bậc (giá trị nhỏ hơn); += cộng dồn; `-=` trừ phần đã dùng; `break` ngừng khi hết điện. **Lưu ý:** Ví dụ 282.000 VND trong đề Word không khớp bảng giá; code tính đúng giá lũy tiến ra **240.000 VND** cho 120 kWh.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Tính tiền điện theo các bậc *lũy tiến*: 50×1.800 + 50×2.000 + 20×2.500 = 240.000 VND cho 120 kWh. Số 282.000 VND trong Word mâu thuẫn với bảng giá; không âm thầm sửa đề.

Thuật ngữ cần nhớ: electricity bill (hóa đơn điện).

Câu 8

Tạo biến tổng ban đầu bằng 0, sau đó cộng từng giá trị của `range(1,n+1)`.

Vì sao lặp từ 1 đến n? Đề yêu cầu luyện vòng lặp; cộng dồn từng số cho thấy tổng được hình thành như thế nào và dễ kiểm tra bằng ví dụ nhỏ. **Quy trình:** Kiểm tra n dương → đặt tổng bằng 0 → duyệt 1 đến n → cộng mỗi số vào tổng.

```
n, total = 10, 0
for value in range(1, n + 1):
    total += value
print(total)  # 55
```

Cú pháp: `range(1, n + 1)` phải cộng 1 ở biên trên vì range **không lấy cận cuối**; += cập nhật tổng. Có thể kiểm tra bằng `n * (n + 1) // 2` nhưng code chính dùng vòng lặp đúng bài.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: range(1,n+1) lấy cả số n; tổng 1..10 phải là 55. Nếu dùng range(1,n), số n sẽ bị bỏ sót.

Thuật ngữ cần nhớ: sum (tổng).

Câu 9

Mỗi lượt dùng phép chia nguyên cho 10 để bỏ chữ số tận cùng, rồi tăng biến đếm; tiếp tục khi số vẫn lớn hơn 0.

Vì sao chia nguyên liên tục? Mỗi lần bỏ chữ số tận cùng làm số ngắn đi một chữ số; đếm số lần cho biết số chữ số. **Quy trình:** Kiểm tra n nguyên dương → đặt bộ đếm 0 → mỗi vòng tăng đếm, bỏ chữ số cuối → dừng khi số bằng 0.

```
n, count = 12345, 0
while n > 0:
    count += 1
    n //= 10
print(count)  # 5
```

Cú pháp: `//= 10` là chia lấy phần nguyên rồi gán lại, nên 123 thành 12; `while` lặp khi điều kiện còn đúng; += tăng bộ đếm. Code này không dùng cho n=0 nếu bài quy định n phải dương.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Trong while, mỗi lượt bỏ một chữ số bằng phép //10 và tăng biến đếm. Kiểm tra ví dụ 123456 phải được sáu lượt.

Thuật ngữ cần nhớ: while loop (vòng lặp điều kiện).

Câu 10

Số nguyên tố phải lớn hơn 1. Thử ước số từ 2 đến căn bậc hai nguyên; nếu tìm thấy ước thì không nguyên tố.

Vì sao chỉ thử đến căn bậc hai? Nếu một số có ước khác 1 và chính nó thì sẽ có ít nhất một ước không lớn hơn căn bậc hai; thử xa hơn là không cần. **Quy trình:** Loại n <= 1 → thử các ước nguyên từ 2 đến căn bậc hai của n → gặp ước chia hết thì trả False → không gặp thì trả True.

```
from math import isqrt
n = 29
prime = n > 1
for divisor in range(2, isqrt(n) + 1):
    if n % divisor == 0:
        prime = False
        break
```

Cú pháp: `isqrt(n)` trả căn bậc hai **nguyên** không vượt quá n; `+1` trong range để xét cả biên; `%` lấy dư, `== 0` nghĩa là chia hết; `break` dừng ngay khi tìm thấy ước. Với n=1 phải trả False.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Số nguyên tố phải lớn hơn 1; thử các ước tới căn bậc hai để tránh kiểm tra thừa. Số 17 là số nguyên tố.

Thuật ngữ cần nhớ: prime (nguyên tố).

Câu 11

Thực hiện thuật toán Euclid bằng phần dư; thay cặp số cho đến khi số thứ hai bằng 0.

Vì sao dùng phép dư? Thuật toán Euclid dựa trên thực tế hai số có chung các ước như cặp số gồm số thứ hai và phần dư khi chia số thứ nhất cho số thứ hai. **Quy trình:** Kiểm tra hai số dương → thay cặp (a,b) bằng (b, phần dư) → tiếp tục đến khi b bằng 0 → a lúc đó là ước chung lớn nhất.

```
a, b = 24, 18
while b != 0:
    a, b = b, a % b
print(a)  # 6
```

Cú pháp: `%` là chia lấy dư; `a, b = b, a % b` cập nhật đồng thời hai biến; `while` ngừng khi b bằng 0. Không phải phép chia `/`, vì cần số dư nguyên.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Thuật toán Euclid thay (a,b) bằng (b,a%b) cho tới khi b=0; với 24 và 36 kết quả là 12.

Thuật ngữ cần nhớ: GCD (ước chung lớn nhất).

Câu 12

Vòng ngoài tạo hàng, vòng trong nối từng dấu *; ghép các hàng bằng xuống dòng.

Vì sao dùng vòng lặp lồng nhau? Hình chữ nhật có số hàng và số cột cố định; vòng ngoài tạo hàng, vòng trong tạo từng dấu sao. **Quy trình:** Kiểm tra hàng/cột dương → với mỗi hàng, nối đúng số dấu sao bằng vòng lặp trong → gom các dòng → ghép bằng ký tự xuống dòng.

```
rows, cols = 2, 3
lines = []
for _ in range(rows):
```

```
    line = ""
    for _ in range(cols):
        line += "*"
    lines.append(line)
result = "\n".join(lines)
```

Cú pháp: `_` là biến lặp không cần dùng; `range(rows)` lặp đúng số hàng; `+=` nối chuỗi dấu sao; `"\n".join(lines)` ghép các dòng xuống hàng. Tổng số dấu sao có thể đối chiếu bằng `rows * cols`, nhưng code thực hành vẫn dùng vòng lặp.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Vòng ngoài tạo m hàng, vòng trong tạo n dấu sao mỗi hàng; ví dụ 4×6 phải có 24 ký tự sao.

Thuật ngữ cần nhớ: nested loop (vòng lặp lồng nhau).

Câu 13

Vòng ngoài chọn số hàng `i`, vòng trong lặp lại ký tự `str(i)` đúng `i` lần.

Vì sao số ký tự tăng theo hàng? Tam giác yêu cầu hàng 1 một ký tự, hàng 2 hai ký tự,... nên vòng trong phải phụ thuộc chỉ số hàng hiện tại, không dùng một số cột cố định. **Quy trình:** Duyệt hàng từ 1 đến n → chuyển số hàng thành chuỗi → lặp đúng i lần để ghép ký tự → lưu dòng.

```
n, rows = 4, []
for i in range(1, n + 1):
    line = ""
    for _ in range(i):
        line += str(i)
    rows.append(line)
```

Cú pháp: `str(i)` chuyển số thành chữ để ghép chuỗi, `range(i)` lặp i lần, `append` đưa dòng vào list. Số ký tự toàn tam giác đối chiếu bằng `n * (n + 1) // 2`.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Dòng thứ i phải lặp chữ số i đúng i lần; dòng thứ 5 là 55555.

Thuật ngữ cần nhớ: triangle (tam giác).

Câu 14

Biến `current` bắt đầu từ 1; vòng trong in số rồi tăng biến này trước khi sang hàng khác.

Vì sao cần biến current? Đề yêu cầu số tăng liên tục qua nhiều hàng, không bắt đầu lại từ 1 mỗi khi sang hàng mới. **Quy trình:** Khởi tạo `current=1` → hàng i tạo đúng i số → sau mỗi số tăng current → ghép số trong hàng bằng dấu cách → ghép các hàng bằng xuống dòng.

```
n, current, rows = 4, 1, []
for i in range(1, n + 1):
    values = []
    for _ in range(i):
        values.append(str(current))
        current += 1
    rows.append(" ".join(values))
result = "\n".join(rows)
```

Cú pháp: `str` chuyển số sang chuỗi; `" ".join(values)` ghép với dấu cách; `+=` tăng số sau mỗi lần in; `"\n".join(rows)` xuống hàng. Số cuối cần bằng `n * (n + 1) // 2`: n=4 cho số cuối 10.

Điểm cần kiểm tra: vòng lặp lồng nhau tạo đúng số dòng/cột và số chữ số.

Ví dụ và lỗi thường gặp: Không đặt lại biến đếm khi xuống hàng: với n=4, hàng cuối phải là 7 8 9 10.

Thuật ngữ cần nhớ: consecutive (liên tiếp).